

signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-29 (578d95c+수정중) → 2026-08-29 · 9744592+수정중 | 4개 파일 · +4 / -8 줄 · 코드 22쪽 | 새 코드북 69쪽

바뀐 내용

· 결과 화면의 '스펙트럼·위터폴' 카드에서 위터폴을 뺐다 — 전체 흐름은 위의 버스트 지도가 이미 보여주므로, 카드는 파워 스펙트럼 하나만 파란 곡선으로 그린다. 표시 칸을 512로 늘리고 최대값만 남기는 방식이라 버스트에서 솟는 좁은 바늘이 반드시 한 칸짜리 스파이크로 보인다.

☐ <code>signus/spectrum.py</code>	+1	-1	2쪽	코드북 9쪽
☐ <code>signus/pipeline.py</code>	+0	-1	2쪽	코드북 31쪽
☐ <code>signus/web/index.html</code>	+1	-2	10쪽	코드북 43쪽
☐ <code>signus/web/app.js</code>	+2	-4	13쪽	코드북 51쪽

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 몇 줄 사라짐 · **↵** = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/spectrum.py +1 -1 · 코드북 9쪽 · 새로 쓸 줄 11

```

1 1 """Spectrum + spectrogram-strip views (display only, never used for detection)."""
2 2
3 3 import numpy as np
4 4 from scipy.signal import stft, welch
5 5
6 6
7 7 def _db(p: np.ndarray) -> np.ndarray:
8 8     return 10 * np.log10(p + 1e-20)
9 9
10 10
11 + def spectrum(x: np.ndarray, fs: float, bins: int = 512) -> dict:
12 12     """sa 프로브의 PSD 판과 같은 조판: 실수부 welch(nperseg<=2048), 단축 0..fs/2 (kHz).
13 13     표시 칸으로 줄일 때는 최대 폴링 -- 좁은 바늘이 살아남게."""
14 14     xd = x.real if np.iscomplexobj(x) else x
15 15     f, p = welch(xd, fs=fs, nperseg=int(min(2048, max(64, xd.size))))
16 16     p = _db(p)
17 17     if f.size > bins: # max-pool so narrow tones survive decimation
18 18         k = f.size // bins
19 19         f, p = f[:k * bins].reshape(bins, k).mean(1), p[:k * bins].reshape(bins, k).max(1)
20 20     return {"f": np.round(f / 1e3, 3).tolist(), "db": np.round(p, 2).tolist()}
21 21
22 22
23 23 def strip(x: np.ndarray, fs: float, max_cols: int = 1600) -> dict:
24 24     """sa 프로브의 PNG 스펙트로그램 띠와 같은 조판의 회색조 이미지 (0..235, row-major,
25 25     위 행 = +fs/2 쪽). hamming 256/128 STFT, 바닥 p25 + 대비폭 10..35 dB 클램프, 열은
26 26     최대 폴링으로만 줄인다 -- 보간이 짧은 버스트를 뭉개지 않게."""
27 27     xd = x.real if np.iscomplexobj(x) else x
28 28     nper = int(min(256, max(64, 1 << int(np.log2(max(xd.size // 2, 64)))))
29 29     _, _, z = stft(xd, fs=fs, window="hamming", nperseg=nper, noverlap=nper // 2,
30 30     return_onesided=True, boundary=None, padded=False)
31 31     db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
32 32     lo = float(np.percentile(db, 25))
33 33     rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo)))
34 34     g = np.clip((db - lo) / rg, 0, 1)[::-1]
35 35     kp = max(1, g.shape[1] // max_cols)
36 36     g = g[:, :g.shape[1] // kp * kp].reshape(g.shape[0], -1, kp).max(2)
37 37     return {"rows": int(g.shape[0]), "cols": int(g.shape[1]),
38 38     "g": np.round(g * 235).astype(int).ravel().tolist()}

```

signus/pipeline.py +0 -1 · 코드북 31쪽 · 새로 쓸 줄

```

1 1 """End-to-end blind demodulation. Two families:
2 2     linear (PSK/QAM): front-end -> carrier -> baud -> matched filter -> timing ->
3 3     classify -> fine sync -> [equalizer rescue] -> quality
4 4     fsk (CPFSK/MSK): frequency discriminator (gated by constant envelope)
5 5     Classification runs BEFORE fine sync so the decision-directed loop knows the
6 6     constellation. Differential demapping is an option, not a detection: dqpsk and
7 7     qpsk share a constellation."""
8 8
9 9 import json
10 10 from dataclasses import dataclass, field
11 11
12 12 import numpy as np
13 13 from scipy.signal import firwin, resample_poly
14 14
15 15 from . import classify as cl
16 16 from . import dsp
17 17 from .chirp import _bw99, analyze_chirp, is_chirp, sweeps_band
18 18 from .constellations import demap_bits, demap_diff_bits, mod_order
19 19 from .eq import equalize, equalize_fse

```

```

20 20 from .fsk import analyze_fsk, fsk_gate
21 21 from .sigio import Meta, parse_name, read
22 22 from .spectrum import spectrum, strip
23 23 from .sync import find_preamble
24 24
25 25 _BITS_CAP = 65536
26 26 _EQ_LOCK = 60.0 # below this a linear signal is worth an equalizer attempt
27 27 _EQ_GAIN = 3.0 # ...keep it only if lock improves by this much
28 28 _EQ_FLOOR = 50.0 # ...and clears this floor, so a wrong-mod fit is never locked in
29 29 _EQ_QAM_FLOOR = 60.0 # ...but a DENSE-QAM (32/64) eq fit must clear a higher bar: the DD loop c
↳ ↳ an
30 30 # drag a lower-order cloud onto a 32/64 lattice at a marginal lock (~51)
↳ ↳ , a
31 31 # confident WRONG order. Genuine multipath recoveries clear ~72; the sweep
↳ ↳ 's
32 32 # eq cases are qpsk/16qam (never 32/64 via eq) so this bar is byte-identic
↳ ↳ al.
33 33 _BAUD_GAIN = 5.0 # an in-band baud hypothesis must beat the global one by this lock
34 34 _SHORT_LOCK = 60.0 # below this, retry the baud line with fewer blocks (short-burst rescue)
35 35 _SYNC_LOCK = 60.0 # below this, try a repeated-preamble carrier/timing sync (packetized bursts
↳ ↳ )
36 36 _ALIAS_MARGIN = 10.0 # a preamble carrier alias must beat the next by this lock, else ambiguous
↳ ↳ .
37 37 # This is the LOAD-BEARING rotation guard: even when the coarse anchor itse
↳ ↳ lf
38 38 # aliases and validates a rotation at ~80 lock, that rotation wins by a thi
↳ ↳ n
39 39 # margin (~8) -- so a 10-lock margin refuses it. Because the margin catche
↳ ↳ s it,
40 40 # the lock floor can stay modest (78, not 82) -> ~5% more genuine recoverie
↳ ↳ s.
41 41 _SYNC_ACCEPT = 78.0 # ...AND clear this absolute lock: below it correct/rotated aliases overlap
↳ ↳ . A
42 42 # 65 floor was tried (to recover more moderate bursts) but an independent
43 43 # snr-12 grid found 8psk rotated aliases locking 71-72 in the opened 65-7
↳ ↳ 8 band
44 44 # (carrier off by one baud/L step -> confident garbage bits). Even 78 leak
↳ ↳ s a
45 45 # a few hard-corner rotations (a separate pre-existing gate limit): hold a
↳ ↳ t 78.
46 46 _SYNC_ADIST = 1.5 # ...AND sit within this many alias steps of the coarse est_carrier fc -- a
47 47 # soft backstop to the margin: it catches the one rotation the margin misse
↳ ↳ s,
48 48 # but tuned LOOSE (1.5, not 0.75) so it does not needlessly reject genuine
49 49 # large-offset recoveries. The three together give 0 confident-wrong across
50 50 # 1920 hard-corner bursts at the max recall this gate reaches.
51 51 _DIFF_OF = {"bpsk": "dbpsk", "qpsk": "dqpsk"} # pi4dqpsk arrives as qpsk -> dqpsk
52 52
53 53
54 54 @dataclass
55 55 class Result:
56 56     meta: Meta
57 57     family: str # 'linear' | 'fsk' | 'chirp'
58 58     burst: tuple[int, int]
59 59     fc: float
60 60     baud: float
61 61     mod: str
62 62     lock: float
63 63     symbols: np.ndarray # aligned symbols (linear) / normalized levels (fsk)
64 64     bits: np.ndarray

```

```

65 65 iq_corr: np.ndarray # exportable corrected baseband
66 66 burst_x: np.ndarray = field(repr=False, default=None) # for spectrum views
67 67 symmetry: int = 0
68 68 carrier_ambiguous: bool = False
69 69 baud_conf: float = 0.0
70 70 rolloff: float | None = None
71 71 h: float | None = None # FSK modulation index
72 72 evm: float = float("nan")
73 73 mer_db: float = float("nan")
74 74 occupied: int = 0
75 75 snr_est: float = float("nan")
76 76 eq_applied: bool = False
77 77 eq_mode: str | None = None # 'sym' | 'fse'
78 78 alias_resolved: bool = False
79 79 baud_fallback: bool = False
80 80 bursts: list = field(default_factory=list)
81 81 burst_idx: int = 0
82 82 preamble: object = None # sync.Preamble when a repeated preamble drove the decode
83 83 chirp: dict | None = None # chirp/CSS characterization (family 'chirp': no symbols)
84 84
85 85 def to_json(self, max_points: int = 6000, views: bool = True) -> dict:
86 86     z = np.nan_to_num(self.symbols) # a dead/DC capture -> NaN symbols -> invalid JSON
87 87     if z.size > max_points: # keep TIME ORDER: the UI animates this sequence
88 88         z = z[np.linspace(0, z.size - 1, max_points).astype(int)]
89 89     det = {"fc": round(self.fc, 3), "baud": round(self.baud, 3), "mod": self.mod,
90 90         "symmetry": self.symmetry, "carrier_ambiguous": self.carrier_ambiguous,
91 91         "baud_conf": round(self.baud_conf, 1),
92 92         "alias_resolved": self.alias_resolved,
93 93         "baud_fallback": self.baud_fallback}
94 94     det["rolloff"] = None if self.rolloff is None else round(self.rolloff, 3)
95 95     rf = self.meta.rf_center # real RF = capture centre + baseband fc
96 96     det["rf_hz"] = None if rf is None else round(rf + self.fc, 3)
97 97     if self.h is not None:
98 98         det["h"] = round(self.h, 3)
99 99     if self.preamble is not None: # a repeated preamble drove the sync
100 100         p = self.preamble
101 101         det["preamble"] = {"period": p.period, "cfo_hz": round(p.cfo_hz, 1),
102 102             "conf": round(p.conf, 2), "start": p.start, "end": p.end}
103 103     if self.chirp is not None: # chirp/CSS characterization -- no constellation fields
104 104         c = self.chirp
105 105         det["chirp"] = {"mu": _r(c["mu"], 1), "up": bool(c["up"]), "bw": _r(c["bw"], 1),
106 106             "sf": c["sf"], "rs": _r(c["rs"], 1), "tsym": _r(c["tsym"], 6)}
107 107     doc = {
108 108         "fs": self.meta.fs, "fmt": self.meta.fmt, "dtype": self.meta.dtype, # dtype
109 109         "rf_center": self.meta.rf_center, # 보고에 필수: lock 0 의 1순위 용의자다
110 110
111 111         "family": self.family, "n_samples": int(self.iq_corr.size),
112 112         "burst": {"start": self.burst[0], "end": self.burst[1]},
113 113         "bursts": [{"start": s, "end": e} for s, e in self.bursts],
114 114         "burst_idx": self.burst_idx,
115 115         "detected": det,
116 116         "quality": {"lock": _r(self.lock, 1) or 0.0, "mer_db": _r(self.mer_db),
117 117             "evm": _r(self.evm, 4), "occupied": int(self.occupied)},
118 118         "snr_est_db": _r(self.snr_est),
119 119         "eq": {"applied": self.eq_applied, "mode": self.eq_mode},
120 120         "constellation": {"i": np.round(z.real, 4).tolist(),
121 121             "q": np.round(z.imag, 4).tolist()},
122 122         "bits": "".join(map(str, self.bits[:_BITS_CAP])),
123 123     }

```

```

124 124         if views and self.burst_x is not None:
125 125             doc["spectrum"] = spectrum(self.burst_x, self.meta.fs)
127 126         return doc
128 127
129 128     def save_iq(self, path: str) -> None:
130 129         np.column_stack([self.iq_corr.real, self.iq_corr.imag]).astype("<f4").tofile(path)
131 130
132 131     def save_symbols(self, path: str) -> None:
133 132         np.save(path, self.symbols.astype(np.complex64))
134 133
135 134     def save_bits(self, path: str, packed: bool = False) -> None:
136 135         if packed:
137 136             np.packbits(self.bits).tofile(path)
138 137         else:
139 138             with open(path, "w") as fh:
140 139                 fh.write("".join(map(str, self.bits)))
141 140
142 141     def save_report(self, path: str) -> None:
143 142         with open(path, "w") as fh:
144 143             json.dump(self.to_json(), fh, indent=1)
145 144
146 145
147 146     def _r(v: float, nd: int = 2) -> float | None:
148 147         return None if v is None or not np.isfinite(v) else round(float(v), nd)
149 148
150 149
151 150     def _fine(z: np.ndarray, mod: str) -> np.ndarray:
152 151         """Bulk-phase removal, decision-directed carrier loop, final rotation lock."""
153 152         return cl.align(dsp.ddsync(cl.align(z, mod), mod), mod)
154 153
155 154
156 155     def _blocks(n: int) -> int:
157 156         """M-th-power spectra are averaged INCOHERENTLY, so a weak tone (high symmetry,
158 157         low SNR) wants fewer, longer segments; only long records need more for
159 158         stationarity. Worth ~2 dB at the 8PSK edge over a fixed blocks=4."""
160 159         return int(np.clip(n // 30000, 1, 8))
161 160
162 161
163 162     def _demod(xd: np.ndarray, fs: float, baud: float, symmetry: int) -> tuple:
164 163         """Matched filter + timing + classify + fine sync at one baud hypothesis."""
165 164         alpha = dsp.est_rolloff(xd, fs, baud)
166 165         ym = dsp.matched(dsp.to_sps(xd, fs, baud), 4, alpha)
167 166         raw = dsp.timing(ym, 4)
168 167         mod = cl.classify(raw, symmetry)
169 168         # align BEFORE ddsync: the coarse carrier leaves only micro-Hz residual, so removing
170 169         # the bulk phase first kills the loop transient that would corrupt the first symbols
171 170         syms = _fine(raw, mod)
172 171         return alpha, ym, raw, mod, syms, cl.quality(syms, mod)
173 172
174 173
175 174     def _rescue(eq_fn, z: np.ndarray, seed_mod: str, symmetry: int) -> tuple:
176 175         """Equalize with a seed constellation, then let the OPENED eye pick the mod:
177 176         heavy ISI corrupts the ring test and even the 2/8 symmetry vote, so re-classify
178 177         with the estimated symmetry AND the neutral order-4 gate, keep the best lock."""
179 178         z1 = eq_fn(z, seed_mod)
180 179         cand_s = []
181 180         for m in {cl.classify(z1, symmetry), cl.classify(z1, 4)}: # symmetry too, per the docstrin
182 181             g
183 182             s = _fine(z1 if m == seed_mod else eq_fn(z, m), m)
183 182             cand_s.append((cl.quality(s, m), s, m))

```

```

184 183 best = max(cands, key=lambda c: c[0].lock)
185 184 # Occam de-fold for the PSK point-doubling: CMA is modulus-only, so a bpsk signal under a
186 185 # multipath echo ALSO fits a qpsk ring at a marginally higher lock (phantom 2->4 points).
187 186 # A bpsk candidate that clears the accept floor cannot be an over-fit of qpsk, so prefer it.
188 187 # Only fires when symmetry==2 put bpsk in the set; a genuine qpsk reads symmetry 4 -> the
189 188 # candidate set is the qpsk singleton -> this branch is unreachable and it stays untouched.
190 189 lo = [c for c in cands if c[2] == "bpsk" and c[0].lock >= _EQ_FLOOR]
191 190 if lo and best[2] == "qpsk":
192 191     return lo[0]
193 192 return best
194 193
195 194
196 195 def analyze(x: np.ndarray, meta: Meta, diff: bool = False,
197 196     burst: int | None = None) -> Result:
198 197     """Run the blind chain. `diff` demaps phase transitions (D-BPSK/D-QPSK);
199 198     `burst` selects one detected burst (default: the most energetic)."""
200 199     if x.size:
201 200         # pathological UNIFORM scale must be fixed BEFORE analytic(): its magnitude sanitizer
202 201         # zeroes samples above 1e150 sample-WISE, so a whole capture near/over that ceiling was
203 202         # wiped wholesale (garbage mods, lock=nan at 1e154) before the rms-normalize below could
204 203         # ever help. The 99th percentile is robust to spike outliers (a single corrupt 1e300
205 204         # sample leaves it at signal scale -> untouched here, still zeroed sample-wise later);
206 205         # normal captures sit far inside (1e-100, 1e100) -> untouched -> sweep byte-identical.
207 206         scale = float(np.percentile(np.abs(x[:16]), 99)) # strided: scale detection needs the
208 207         # BULK magnitude, not every sample -- 16x cheaper on a large direct capture
209 208         if np.isfinite(scale) and scale > 0 and not (1e-100 < scale < 1e100):
210 209             x = x / scale
211 210     x = dsp.analytic(x)
212 211     x = x - x.mean() # DC block: LO leakage otherwise corrupts every estimator
213 212     if x.size < 64: # empty / trivially short: nothing to estimate, and it crashes downstream
214 213         raise ValueError(f"신호가 너무 짧습니다 ({x.size} 샘플)")
215 214     rms = float(np.sqrt(np.mean(np.abs(x) ** 2)))
216 215     if rms and (rms < 1e-6 or rms > 1e6): # only PATHOLOGICAL scale: normalize so the scale-var
217 216         x = x / rms # spots (fsk_gate's CV epsilon, est_carrier's x**p ov
218 217         # er/
219 218         # underflow) stay invariant. Normal captures rms~0(1
220 219         # untouched -> the acceptance sweep is byte-identical
221 220         .
222 221     bursts = dsp.find_bursts(x, meta.fs)
223 222     if burst is None or not 0 <= burst < len(bursts):
224 223         burst = int(np.argmax([np.sum(np.abs(x[s:e]) ** 2) for s, e in bursts]))
225 224     s, e = bursts[burst]
226 225     xb = x[s:e] if e - s >= 64 else x
227 226
228 227     # chirp gate: an FMCW/CSS sweep trips fsk_gate (bimodal IF) and would force-demodulate
229 228     # into confident garbage FSK. sweeps_band keeps real FSK (0/222 pass) out of this branch,
230 229     # so only a genuine band-sweep lands here -- characterized, never constellation-demodulated.
231 230     # The gates read a band-ISOLATED copy: mix to the spectral centroid, low-pass to the
232 231     # 99%-power band, and decimate so the signal fills ~28% of Nyquist. An off-centre
233 232     # narrowband sweep in a wide record otherwise dilutes the IF statistics with
234 233     # out-of-band noise and leaks through as garbage FSK.
235 234     pw = np.abs(np.fft.fft(xb)) ** 2 # sweep centre = spectral centroid
236 235     fr = np.fft.fftfreq(xb.size, 1 / meta.fs) # (est_carrier means nothing on a mover)
237 236     fcg = float(np.sum(fr * pw) / (np.sum(pw) + 1e-30))
238 237     xg = dsp.mix(xb, meta.fs, fcg)
239 238     bwg = _bw99(xg, meta.fs)
240 239     want = max(1.25 * bwg, 1e-4 * meta.fs)
241 240     d = int(np.clip(0.28 * meta.fs / want, 1, max(1, xg.size // 256)))

```

```

240 239 fsg = meta.fs / d
241 240 xg = np.convolve(xg, firwin(129, max(min(0.9 * fsg / 2, bwg), 1e-4 * meta.fs)
242 241 / (meta.fs / 2)), "same")
243 242 if d > 1:
244 243 xg = resample_poly(xg, 1, d)
245 244 if is_chirp(xg, fsg) and sweeps_band(xg, fsg):
246 245 info = analyze_chirp(xg, fsg)
247 246 return Result(meta, "chirp", (s, e), fcg, info["rs"] or 0.0,
248 247 "lora" if info["sf"] else "chirp", 0.0, np.zeros(0, complex),
249 248 np.zeros(0, np.uint8), x, burst_x=xb,
250 249 bursts=bursts, burst_idx=burst, chirp=info)
251 250
252 251 if fsk_gate(xb, meta.fs):
253 252 r = analyze_fsk(xb, meta.fs)
254 253 sym = np.asarray(r["symbols"], dtype=np.complex128)
255 254 return Result(meta, "fsk", (s, e), r["fc"], r["baud"], r["mod"], r["lock"],
256 255 sym, r["bits"], xb, burst_x=xb, h=r["h"],
257 256 bursts=bursts, burst_idx=burst)
258 257
259 258 fc0, symmetry, _ = dsp.est_carrier(xb, meta.fs, blocks=_blocks(xb.size))
260 259 fc = dsp.resolve_alias(xb, meta.fs, fc0, symmetry)
261 260 amb = abs(symmetry * fc) > 0.4 * meta.fs
262 261 xd = dsp.mix(xb, meta.fs, fc)
263 262
264 263 baud, conf = dsp.est_baud(xd, meta.fs)
265 264 fell = False
266 265 alpha, ym, raw, mod, syms, q = _demod(xd, meta.fs, baud, symmetry)
267 266 bw = dsp.occupied_bw(xd, meta.fs)
268 267 if bw and not (0.72 * bw <= baud <= 1.05 * bw):
269 268 # the global |x|^2 peak disagrees with the occupied band: below alpha~0.1
270 269 # (and under harsh channels) data junk outruns the true line. Demod the
271 270 # in-band hypotheses too and let lock decide, with a clear margin.
272 271 for lo in (0.80, 0.67):
273 272 b2, c2 = dsp.est_baud(xd, meta.fs, lo=lo * bw, hi=1.02 * bw)
274 273 if abs(b2 - baud) <= 0.01 * b2:
275 274 continue
276 275 t = _demod(xd, meta.fs, b2, symmetry)
277 276 if t[5].lock > q.lock + _BAUD_GAIN:
278 277 alpha, ym, raw, mod, syms, q = t
279 278 baud, conf, fell = b2, c2, True
280 279
281 280 if q.lock < _SHORT_LOCK:
282 281 # short / low-SNR burst rescue: the default 4-block |x|^2 line splits the record
283 282 # too finely and locks a junk peak. Fewer, longer blocks give a stronger line (the
284 283 # carrier_blocks logic, applied to baud). Run both and keep only a clearly-better
285 284 # lock -- so a long, already-locked signal (CORE) never enters here and is untouched.
286 285 for nb in (2, 1):
287 286 b2, c2 = dsp.est_baud(xd, meta.fs, blocks=nb)
288 287 if abs(b2 - baud) <= 0.01 * b2:
289 288 continue
290 289 t = _demod(xd, meta.fs, b2, symmetry)
291 290 if t[5].lock > q.lock + _BAUD_GAIN:
292 291 alpha, ym, raw, mod, syms, q = t
293 292 baud, conf, fell = b2, c2, True
294 293
295 294 eq_applied, eq_mode, pre = False, None, None
296 295 if q.lock < _EQ_LOCK: # ISI (multipath) is what a phase-only loop cannot fix
297 296 # CMA is modulus-only, so it opens the eye without knowing the constellation.
298 297 qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
299 298 mode = "sym"

```

```

300 299     if qe.lock < max(_EQ_FLOOR, q.lock + _EQ_GAIN):
301 300         # symbol-spaced FIR could not invert the channel; retry T/2-spaced on
302 301         # the clock-tracked 2 sps stream (unaligned spectrum, longer inverse)
303 302         q2, s2, m2 = _rescue(lambda z, m: equalize_fse(z, m),
304 303                                dsp.timing(ym, 4, out=2), mod, symmetry)
305 304         if q2.lock > qe.lock:
306 305             qe, se, me, mode = q2, s2, m2, "fse"
307 306         floor = _EQ_QAM_FLOOR if me in ("32qam", "64qam") else _EQ_FLOOR
308 307         if qe.lock > q.lock + _EQ_GAIN and qe.lock >= floor:
309 308             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
310 309     elif mod in ("16qam", "32qam", "64qam"):
311 310         # confident square-QAM at high lock: a benign post-echo can fold a LOWER-order signal
312 311         # onto a QAM lattice with a clean-looking eye, so the low-lock rescue above never fires.
313 312         # Equalize once and accept ONLY if a strictly lower order emerges -- verified never to
314 313         # demote a genuine QAM (0/36 across 16/32/64qam x snr x seeds), so real QAM is untouched
315 314         qe, se, me = _rescue(lambda z, m: equalize(z, m), raw, mod, symmetry)
316 315         mode = "sym"
317 316         if mod_order(me) < mod_order(mod) and qe.lock < _EQ_FLOOR:
318 317             # a lower order emerged but the symbol-spaced eye stayed shut (echo > ~2 symbols);
319 318             # retry T/2 fractionally-spaced. Only runs once a de-fold is INDICATED, so a genuine
320 319             # QAM (no lower order) never pays for the FSE.
321 320             qe, se, me = _rescue(lambda z, m: equalize_fse(z, m),
322 321                                dsp.timing(ym, 4, out=2), mod, symmetry)
323 322             mode = "fse"
324 323         if mod_order(me) < mod_order(mod) and qe.lock >= _EQ_FLOOR:
325 324             syms, mod, q, eq_applied, eq_mode = se, me, qe, True, mode
326 325
327 326     if q.lock < _SYNC_LOCK:
328 327         # packetized-burst rescue -- LAST, after the equalizer: a repeated preamble pins the
329 328         # carrier/ baud/ timing from only a few symbols, where the blind M-th-power estimator (nee
330 329         # many symbols to average) fails. Runs only if the eq rescue above did NOT lift the lock
331 330         # a multipath signal (which the eq handles, and whose ISI can fake a short period) never
332 331         # reaches here. Detect the preamble, mix by its CFO, demod the DATA past it, keep-best --
333 332         # random / no-preamble signal yields no plateau or a CFO that does not improve lock an
334 333         # dropped (the sweep stays byte-identical). The preamble period P = L*sps supplies the b
335 334         # (fs*L/P per block length L), the phase slope gives the carrier (+-fs/P resolves the L-
336 335         # alias); every PSK/QAM symmetry is tried -- the right (carrier, baud, sym) locks clean.
337 336         ps = find_preamble(xb, meta.fs, baud_hint=baud)
338 337         if ps is not None:
339 338             # The Schmidl-Cox CFO is ambiguous modulo fs/P = baud/L: an M-PSK carrier off by bau
340 339             # rotates a whole constellation position per symbol, so the eye still looks locked b
341 340             # the bits shift -> a confident WRONG decode if the wrong alias is trusted. The coar
342 341             # M-th-power fc lands within ~1 alias step of the truth even on these short bursts
343 342             # CENTRE the alias scan on it (k0) and sweep +-3 steps -- this reaches the correct a
344 343             # for any offset (not just |fc|<baud/2); keep-best over them picks the cleanest eye.
345 344             step = meta.fs / ps.period
346 345             k0 = round((fc - ps.cfo_hz) / step)
347 346             cands = []

```

```

348 347     for b2 in sorted({round(meta.fs * L / ps.period) for L in (3, 4, 6, 8)}):
349 348         for k in range(k0 - 3, k0 + 4):
350 349             cfo = ps.cfo_hz + k * step
351 350             data = dsp.mix(xb, meta.fs, cfo)[ps.end:]
352 351             if data.size < 256:
353 352                 continue
354 353             for sm in (2, 4, 8):
355 354                 t = _demod(data, meta.fs, float(b2), sm)
356 355                 cand.s.append((t[5].lock, cfo, float(b2), sm, t))
357 356             cand.s.sort(key=lambda c: -c[0])
358 357             # Adjacent aliases both look like clean M-PSK, so lock alone can pick the WRONG on
359 358             # e by a
360 359             # hair. Require the winner to beat the best DIFFERENT-carrier candidate by a clear g
361 360             # ap;
362 361             # otherwise the alias is genuinely ambiguous -> leave the honest low-lock result rat
363 362             # her
364 363             # than emit a confident wrong decode. (Same-carrier baud/sym variants do not compete
365 364             # .)
366 365             # PRECISION GATE: the alias ambiguity (carrier off by baud/L) is blind-ambiguous -
367 366             # - a
368 367             # rotated alias still locks AS HIGH AS the truth (both are clean M-PSK), so lock alo
369 368             # ne
370 369             # cannot separate them (a rotated 8psk alias was measured locking 78 with ber 0.38)
371 370             # . Two
372 371             # conjunctive conditions favour precision over recall: (1) a strong absolute lock, a
373 372             # nd
374 373             # (2) the winner sits within _SYNC_ADIST alias steps of the coarse est_carrier fc -
375 374             # - a
376 375             # baud/L rotation is >=1 step off the true carrier, so a far-from-anchor winner is a
377 376             # rotation. Genuine recoveries have a near-anchor carrier (adist<=0.5, lock 88+); o
378 377             # n the
379 378             # hardest bursts the anchor itself is unreliable -> both conditions fail -> refuse.
380 379             if cand.s and cand.s[0][0] >= _SYNC_ACCEPT and cand.s[0][0] > q.lock + _EQ_GAIN:
381 380                 top = cand.s[0]
382 381                 other = next((c for c in cand.s if abs(c[1] - top[1]) > step / 2), None)
383 382                 anchored = abs(top[1] - fc) <= _SYNC_ADIST * step
384 383                 if anchored and (other is None or top[0] - other[0] >= _ALIAS_MARGIN):
385 384                     alpha, ym, raw, mod, syms, q = top[4]
386 385                     fc, baud, symmetry = top[1], top[2], top[3]
387 386                     eq_applied, eq_mode, pre = False, None, ps
388 387                     # diagnostics must describe the ACCEPTED decode, not the abandoned blind
389 388                     # attempt: the carrier came from the preamble (not resolve_alias -> fc0=fc
390 389                     # keeps alias_resolved False), the baud from the preamble period (no
391 390                     # spectral line -> conf 0, no fallback), and ambiguity follows the new fc.
392 391                     fc0, conf, fell = fc, 0.0, False
393 392                     amb = abs(symmetry * fc) > 0.4 * meta.fs
394 393
395 394             bits = demap_diff_bits(syms, _DIFF_OF[mod]) if diff and mod in _DIFF_OF \
396 395             else demap_bits(syms, mod)
397 396             return Result(meta, "linear", (s, e), fc, baud, mod, q.lock, syms, bits, ym,
398 397             burst_x=xb, symmetry=symmetry, carrier_ambiguous=amb, baud_conf=conf,
399 398             rolloff=alpha, evm=q.evm, mer_db=q.mer_db, occupied=q.occupied,
400 399             snr_est=cl.snr_m2m4(syms, mod), eq_applied=eq_applied, eq_mode=eq_mode,
401 400             alias_resolved=abs(fc - fc0) > 1.0, baud_fallback=fell,
402 401             bursts=bursts, burst_idx=burst, preamble=pre)
403 402
404 403
405 404
406 405
407 406
408 407
409 408
410 409
411 410
412 411
413 412
414 413
415 414
416 415
417 416
418 417
419 418
420 419
421 420
422 421
423 422
424 423
425 424
426 425
427 426
428 427
429 428
430 429
431 430
432 431
433 432
434 433
435 434
436 435
437 436
438 437
439 438
440 439
441 440
442 441
443 442
444 443
445 444
446 445
447 446
448 447
449 448
450 449
451 450
452 451
453 452
454 453
455 454
456 455
457 456
458 457
459 458
460 459
461 460
462 461
463 462
464 463
465 464
466 465
467 466
468 467
469 468
470 469
471 470
472 471
473 472
474 473
475 474
476 475
477 476
478 477
479 478
480 479
481 480
482 481
483 482
484 483
485 484
486 485
487 486
488 487
489 488
490 489
491 490
492 491
493 492
494 493
495 494
496 495
497 496
498 497
499 498
500 499
501 500
502 501
503 502
504 503
505 504
506 505
507 506
508 507
509 508
510 509
511 510
512 511
513 512
514 513
515 514
516 515
517 516
518 517
519 518
520 519
521 520
522 521
523 522
524 523
525 524
526 525
527 526
528 527
529 528
530 529
531 530
532 531
533 532
534 533
535 534
536 535
537 536
538 537
539 538
540 539
541 540
542 541
543 542
544 543
545 544
546 545
547 546
548 547
549 548
550 549
551 550
552 551
553 552
554 553
555 554
556 555
557 556
558 557
559 558
560 559
561 560
562 561
563 562
564 563
565 564
566 565
567 566
568 567
569 568
570 569
571 570
572 571
573 572
574 573
575 574
576 575
577 576
578 577
579 578
580 579
581 580
582 581
583 582
584 583
585 584
586 585
587 586
588 587
589 588
590 589
591 590
592 591
593 592
594 593
595 594
596 595
597 596

```

```

398 397         burst: int | None = None, rf: float | None = None) -> Result:
399 398         """Analyze a file; explicit args override SigMF/filename tokens."""
400 399         from .sigio import parse_sigmf
401 400         m = parse_sigmf(path) or parse_name(path)
402 401         meta = Meta(fs or m.fs, fmt or m.fmt, dtype or m.dtype,
403 402                     endian or m.endian, m.bitrev if bitrev is None else bitrev,
404 403                     rf_center=rf if rf is not None else m.rf_center)
405 404         x, meta = read(path, meta)
406 405         return analyze(x, meta, diff, burst)
407 406
408 407
409 408     # --- web payload -----
410 409
411 410     def analyze_web(x: np.ndarray, meta: Meta, *, burst: int | None = None) -> dict:
412 411         """Payload for POST /api/analyze: the analyze() result as JSON, plus -- on the first
413 412         load of a multi-burst capture -- a whole-record spectrogram strip so the UI can draw
414 413         the burst map. Burst re-selection posts again with ?burst=N and skips the overview
415 414         (the UI keeps the one it already has)."""
416 415         r = analyze(x, meta, burst=burst)
417 416         out = r.to_json()
418 417         if burst is None and len(r.bursts) > 1:
419 418             out["overview"] = {"n": int(x.size), "fs": meta.fs, "strip": strip(x, meta.fs)}
420 419         return out

```

signus/web/index.html +1 -2 · 코드북 43쪽 · 새로 쓸 줄 118

```

1 1     <!doctype html>
2 2     <html lang="ko">
3 3     <head>
4 4     <meta charset="utf-8">
5 5     <meta name="viewport" content="width=device-width, initial-scale=1">
6 6     <title>Signus - 신호 복조 분석기</title>
7 7     <link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='
4 4         0 0 16 16'%3E%3Crect width='16' height='16' rx='5' fill='%233182F6'/%3E%3C/svg%3E">
8 8     <link rel="stylesheet" href="/style.css">
9 9     <script defer src="/app.js"></script>
10 10    </head>
11 11    <body>
12 12    <header class="topbar">
13 13        <div class="brand">
14 14            <span class="logo" aria-hidden="true"></span>
15 15            <div>
16 16                <h1>Signus</h1>
17 17                <p class="tagline">블라인드 PSK/QAM/FSK 신호 복조 분석기</p>
18 18            </div>
19 19        </div>
20 20    </header>
21 21
22 22    <main class="wrap">
23 23        <section id="dropCard" class="card drop" tabindex="0" role="button"
24 24            aria-label="신호 파일 선택 또는 드래그">
25 25            <input id="fileInput" type="file" multiple hidden>
26 26            <div class="drop-icon" aria-hidden="true"></div>
27 27            <p class="drop-title">신호 파일을 여기에 놓으세요</p>
28 28            <p class="drop-sub">여러 개를 놓으면 일괄 분석 · <code>.json</code> 정답 · <code>.sigmf-meta
4 4                </code> 지원</p>
29 29        </section>
30 30
31 31        <section id="metaForm" class="card form hidden">
32 32            <p class="form-title">파일 이름에서 정보를 찾지 못했어요. 직접 입력해주세요.</p>
33 33            <div class="form-row">

```

```

34 34      <label>샘플레이트 (Hz)
35 35      <input id="mFs" type="number" min="1" step="any" placeholder="예: 1000000"></label>
36 36      <label>포맷
37 37      <select id="mFmt">
38 38          <option value="">선택</option><option value="iq">IQ</option><option value="real">Real<
39 39      </select></label>
40 40      <label>데이터 타입
41 41      <select id="mDtype">
42 42          <option value="i16">i16 (16t)</option><option value="i8">i8 (8t)</option>
43 43          <option value="u8">u8 (8o)</option><option value="u16">u16 (16o)</option>
44 44          <option value="f32">f32</option><option value="f64">f64</option>
45 45      </select></label>
46 46  </div>
47 47      <button id="metaGo" class="btn">분석 시작</button>
48 48  </section>
49 49
50 50  <section id="loading" class="card loading hidden" aria-live="polite">
51 51      <div class="spinner" aria-hidden="true"></div>
52 52      <p id="loadMsg">신호를 분석하고 있어요...</p>
53 53  </section>
54 54
55 55  <section id="errorCard" class="card error hidden" role="alert">
56 56      <div class="err-mark" aria-hidden="true">!</div>
57 57      <div>
58 58          <p class="err-title">분석에 실패했어요</p>
59 59          <p id="errMsg" class="err-msg"></p>
60 60      </div>
61 61  </section>
62 62
63 63  <!-- Batch results table -->
64 64  <section id="batchCard" class="card hidden">
65 65      <div class="card-head">
66 66          <h2 class="card-title">일괄 분석 결과</h2>
67 67          <button id="dlCsv" class="btn ghost small">CSV 내보내기</button>
68 68      </div>
69 69      <div class="table-wrap">
70 70          <table class="batch">
71 71              <thead><tr><th>파일</th><th>변조</th><th>중심주파수</th><th>심볼레이트</th><th>Lock</th>
72 72              </tr></thead>
73 73              <tbody id="batchBody"></tbody>
74 74          </table>
75 75      </div>
76 76      <p class="hint">행을 클릭하면 상세 화면으로 이동합니다.</p>
77 77  </section>
78 78
79 79  <section id="results" class="results hidden">
80 80      <button id="backBatch" class="btn ghost small hidden">← 목록으로</button>
81 81
82 82      <div class="card summary">
83 83          <div class="summary-head">
84 84              <span id="statusPill" class="pill"></span>
85 85              <p id="fileName" class="file-name"></p>
86 86          </div>
87 87          <div class="summary-body">
88 88              <div class="gauge">
89 89                  <svg viewBox="0 0 120 120" class="donut" aria-hidden="true">
90 90                      <circle class="donut-track" cx="60" cy="60" r="52"></circle>
91 91                      <circle id="donutArc" class="donut-arc" cx="60" cy="60" r="52"></circle>

```

```

92 92      <div class="gauge-center"><span id="lockNum" class="gauge-num">0</span>
93 93      <span class="gauge-cap tip" title="별자리가 얼마나 조밀·정렬됐는지 (0~100). 60 이상
    ↳ ↳      이면 복조 신뢰">LOCK</span></div>
94 94      </div>
95 95      <div class="metrics">
96 96      <div class="metric"><span class="metric-label tip" title="변조 오차비 - 높을수록 깨
    ↳ ↳      끓 (dB)">MER</span>
97 97      <span id="merVal" class="metric-val"></span></div>
98 98      <div class="metric"><span class="metric-label tip" title="오차 벡터 크기 - 낮을수록 좋
    ↳ ↳      음 (%)>EVM</span>
99 99      <span id="evmVal" class="metric-val"></span></div>
100 100      <div class="metric"><span class="metric-label tip" title="추정 심볼 신호 대 잡음비 (dB
    ↳ ↳      )">SNR</span>
101 101      <span id="snrVal" class="metric-val"></span></div>
102 102      </div>
103 103      </div>
104 104      <div id="flagRow" class="chips"></div>
105 105      <div id="burstRow" class="chips"></div>
106 106      </div>
107 107
108 108      <div id="burstMap" class="card hidden">
109 109      <h2 class="card-title">버스트 지도</h2>
110 110      <div class="wf-wrap">
111 111      <canvas id="burstFall" class="fall"></canvas>
112 112      <div id="burstBoxes" class="boxes"></div>
113 113      </div>
114 114      <p class="hint">전체 녹음의 스펙트로그램 (가로=시간, 위=+주파수) · 위 번호와 아래 띠가 검
    ↳ ↳      출된 버스트. 구간을 클릭하면 그 구간을 분석합니다.</p>
115 115      </div>
116 116
117 117      <div class="card">
118 +      <h2 class="card-title">파워 스펙트럼</h2>
119 119      <canvas id="specCanvas" class="spec"></canvas>
120 120      </div>
121 121
122 122
123 122      <div class="card const-card">
124 123      <div class="card-head">
125 124      <h2 class="card-title">성상도</h2>
126 125      <div class="play-row">
127 126      <button id="playBtn" class="btn ghost small" aria-label="재생/일시정지">⏮ 일시정지</bu
    ↳ ↳      tton>
128 127      <select id="speedSel" class="sel small" aria-label="재생 속도">
129 128      <option value="1">1x</option><option value="4" selected>4x</option>
130 129      <option value="16">16x</option>
131 130      </select>
132 131      </div>
133 132      </div>
134 133      <canvas id="constCanvas" class="const"></canvas>
135 134      <input id="scrub" class="scrub" type="range" min="0" max="1000" value="0"
136 135      aria-label="재생 위치">
137 136      <p id="playInfo" class="hint"></p>
138 137      </div>
139 138
140 139      <div id="paramGrid" class="param-grid"></div>
141 140
142 141      <div class="card downloads">
143 142      <h2 class="card-title">다운로드</h2>
144 143      <div class="dl-row">
145 144      <button id="copyBits" class="btn ghost">비트 복사</button>
146 145      <button id="dlBits" class="btn ghost">비트 .txt</button>

```

```

147 146      <button id="dlSyms" class="btn ghost">심볼 .csv</button>
148 147      <button id="dlReport" class="btn ghost">리포트 .json</button>
149 148    </div>
150 149  </div>
151 150 </section>
152 151 </main>
153 152 </body>
154 153 </html>

```

signus/web/app.js +2 -4 · 코드북 51쪽 · 새로 쓸 줄 437, 446

```

1 1  "use strict";
2 2  const $ = (id) => document.getElementById(id);
3 3  const API = "/api/analyze";
4 4  const FS_RE = /(?:^[_.])fs(\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[_.]|$))/i;
5 5  const RF_RE = /(?:^[_.])rf(\d+(?:\.\d+)?(?:e[+-]?\d+)?(?:[_.]|$))/i;
6 6  const REDUCED = matchMedia("(prefers-reduced-motion: reduce)").matches;
7 7  const state = { file: null, sidecar: null, meta: null, resp: null, batch: [],
8 8                  burst: null, overview: null };
9 9
10 10 /* --- filename parsing (mirror of sigio.parse_name; read-necessities only) --- */
11 11 const DTYPES = ["i8", "u8", "i16", "u16", "f32", "f64"];
12 12 // baudline-style aliases: <bits>t = two's complement (signed), <bits>o = offset (unsigned)
13 13 const DTYPE_ALIAS = { "8t": "i8", "8o": "u8", "16t": "i16", "16o": "u16", "32f": "f32", "64f": "
14 14   f64" };
15 15 const FMTS = { cplx: "iq", real: "real", iq: "iq" }; // iq = 옛 밀줄 형식의 이름
16 16 /* <이름>.cplx|real.<샘플레이트>.<16t>.pcm - 샘플레이트는 접두사가 없으므로 포맷 바로
17 17   다음 자리로 찾는다. 옛 밀줄 형식(<이름>_fs_...iq_i16.iq)도 그대로 읽는다. sigio.py 와 규칙이
18 18   같아야 한다: 여기서 갈리면 웹은 되고 CLI 는 안 되는(또는 그 반대) 조용한 불일치가 된다. */
19 19 function parseName(name) {
20 20   const base = name.replace(/^.*/, "").toLowerCase();
21 21   const mt = FS_RE.exec(base), rt = RF_RE.exec(base), toks = base.split(/[_.]/);
22 22   // 진짜 포맷 토막 = "숫자 + 아는 제원 토막"을 데리고 다니는 마지막 후보. 라벨의 real/cplx/
23 23   // u8/be 오염과 점 낀 샘플레이트(2.4e6 -> 2 Hz)를 같은 관문으로 막는다 (sigio.py 와 동일 규칙)
24 24   const spec = (t) => DTYPES.includes(t) || t in DTYPE_ALIAS || t === "be" || t === "bitrev"
25 25   || t === "pcm" || t.startsWith("rf");
26 26   const hit = (k) => /^d+$/i.test(toks[k + 1] || "") && spec(toks[k + 2] || "");
27 27   const cand = toks.map((t, k) => (t in FMTS ? k : -1)).filter((k) => k >= 0);
28 28   const hits = cand.filter(hit);
29 29   const i = hits.length ? hits[hits.length - 1] : (cand.length ? cand[0] : -1);
30 30   const tail = i >= 0 ? toks.slice(i + 1) : toks; // 제원은 포맷 토막 뒤에만 산다
31 31   const dt = tail.find((t) => DTYPES.includes(t) || t in DTYPE_ALIAS);
32 32   return {
33 33     fs: hits.includes(i) ? parseFloat(toks[i + 1]) : (mt ? parseFloat(mt[1]) : null),
34 34     fmt: i >= 0 ? FMTS[toks[i]] : null,
35 35     dtype: dt ? (DTYPE_ALIAS[dt] || dt) : "i16",
36 36     endian: tail.includes("be") ? "be" : "le",
37 37     bitrev: tail.includes("bitrev"),
38 38     rf: rt ? parseFloat(rt[1]) : null,
39 39   };
40 40 }
41 40 /* SigMF: core:datatype -> our fmt/dtype/endian */
42 41 const SIGMF = { cf32: ["iq", "f32"], ci16: ["iq", "i16"], ci8: ["iq", "i8"],
43 42   cu8: ["iq", "u8"], rf32: ["real", "f32"], ri16: ["real", "i16"] };
44 43 function metaFromSigmf(doc) {
45 44   const g = doc && doc.global;
46 45   if (!g || !g["core:datatype"]) return null;
47 46   const dt = g["core:datatype"], base = dt.replace(/_[lbe]$/, "");
48 47   if (!(base in SIGMF)) return null;
49 48   const [fmt, dtype] = SIGMF[base];

```

```

49 49     const cap = (doc.captures || []).[0] || {};
50 50     return { fs: Number(g["core:sample_rate"]), fmt, dtype,
51 51             endian: dt.endsWith("_be") ? "be" : "le", bitrev: false,
52 52             rf: cap["core:frequency"] != null ? Number(cap["core:frequency"]) : null };
53 53 }
54 54
55 55 /* --- number formatting --- */
56 56 function sig(x) {
57 57     const a = Math.abs(x);
58 58     return parseFloat(a >= 100 ? x.toFixed(0) : a >= 10 ? x.toFixed(1) : x.toFixed(2)).toString();
59 59 }
60 60 function fmtHz(v) {
61 61     if (v == null) return "-";
62 62     const a = Math.abs(v);
63 63     if (a >= 1e6) return sig(v / 1e6) + " MHz";
64 64     if (a >= 1e3) return sig(v / 1e3) + " kHz";
65 65     return sig(v) + " Hz";
66 66 }
67 67 function fmtRf(v) { // absolute RF: keep kHz resolution even in the 100s-of-MHz range
68 68     return (v / 1e6).toFixed(3) + " MHz";
69 69 }
70 70
71 71 /* --- ideal constellation points (mirror of constellations.ideal_points) --- */
72 72 function idealPoints(mod) {
73 73     if (mod === "bpsk") return [{ i: 1, q: 0 }, { i: -1, q: 0 }];
74 74     if (mod === "qpsk" || mod === "8psk") {
75 75         const m = mod === "qpsk" ? 4 : 8, off = mod === "qpsk" ? Math.PI / 4 : 0, p = [];
76 76         for (let k = 0; k < m; k++) p.push({ i: Math.cos(off + 2 * Math.PI * k / m), q: Math.sin(
77 77             off + 2 * Math.PI * k / m ) });
78 78         return p;
79 79     }
80 80     const n = mod === "16qam" ? 4 : mod === "32qam" ? 6 : 8, lv = [], p = [];
81 81     for (let k = 0; k < n; k++) lv.push(k * 2 - (n - 1));
82 82     for (const qi of lv) for (const ii of lv) {
83 83         if (mod === "32qam" && Math.abs(ii * qi) === 25) continue; // cross: corners removed
84 84         p.push({ i: ii, q: qi });
85 85     }
86 86     let e = 0;
87 87     for (const pt of p) e += pt.i * pt.i + pt.q * pt.q;
88 88     const norm = Math.sqrt(e / p.length);
89 89     return p.map((pt) => ({ i: pt.i / norm, q: pt.q / norm }));
90 90 }
91 91 /* --- file intake: one data file (+sidecar) or many (batch) --- */
92 92 async function acceptFiles(list) {
93 93     const files = [...list];
94 94     const metas = files.filter((f) => /\. (json|sigmf-meta)$/i.test(f.name));
95 95     const data = files.filter((f) => !metas.includes(f));
96 96     if (!data.length) return showError("데이터 파일을 찾을 수 없어요.");
97 97     if (data.length > 1) return runBatch(data, metas);
98 98
99 99     state.file = data[0];
100 100     state.resp = null;
101 101     state.batch = [];
102 102     state.burst = null;
103 103     state.overview = null;
104 104     // 사이드카는 이름이 같은 캡처의 것만 쓴다 -- 확장자만 보고 집으면 다른 신호의 fs/fmt 로
105 105     // 조용히 읽는다 (일괄 모드는 이미 stem 대조를 한다: 두 모드가 다르면 그게 또 함정이다)
106 106     const mine = (m, ext) => {
107 107         const stem = m.name.toLowerCase().slice(0, -ext.length);

```

```

108 108     const dn = state.file.name.toLowerCase();
109 109     return dn.startsWith(stem) && (dn.length === stem.length || dn[stem.length] === ".");
110 110 };
111 111 const sm = metas.find((m) => m.name.toLowerCase().endsWith(".sigmf-meta") && mine(m, ".sigmf-m
    4 4     eta"));
112 112 const truth = metas.find((m) => m.name.toLowerCase().endsWith(".json") && mine(m, ".json"));
113 113 state.sidecar = truth ? await readJson(truth) : null; // client-side only, never sent
114 114 state.meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(state.file.name);
115 115 if (state.meta && state.meta.fs && state.meta.fmt) runFirst();
116 116 else showMetaForm();
117 117 }
118 118 const readJson = (f) => f.text().then(JSON.parse).catch(() => null);
119 119
120 120 function showMetaForm() {
121 121     hideAll();
122 122     state.meta = state.meta || {};
123 123     if (state.meta.fs) $("mFs").value = state.meta.fs;
124 124     $("mFmt").value = state.meta.fmt || "";
125 125     $("mDtype").value = state.meta.dtype || "i16";
126 126     show($("metaForm"));
127 127 }
128 128 $("metaGo").onclick = () => {
129 129     const fs = parseFloat($("mFs").value), fmt = $("mFmt").value;
130 130     if (!(fs > 0) || !fmt) return showError("샘플레이트와 포맷을 입력해주세요.");
131 131     state.meta = { fs, fmt, dtype: $("mDtype").value, endian: "le", bitrev: false };
132 132     runFirst();
133 133 };
134 134
135 135 /* --- server call (contract-frozen) --- */
136 136 function query(file, m, burst) {
137 137     const q = new URLSearchParams({ name: file.name });
138 138     if (m.fs) q.set("fs", m.fs);
139 139     if (m.fmt) q.set("fmt", m.fmt);
140 140     if (m.dtype) q.set("dtype", m.dtype);
141 141     if (m.endian) q.set("endian", m.endian);
142 142     if (m.bitrev) q.set("bitrev", "1");
143 143     if (m.rf != null) q.set("rf", m.rf);
144 144     if (burst != null) q.set("burst", burst);
145 145     return q;
146 146 }
147 147 async function postTo(api, file, m, burst) {
148 148     const res = await fetch(api + "?" + query(file, m, burst), { method: "POST", body: file });
149 149     const data = await res.json();
150 150     if (!res.ok || data.error) throw new Error(data.error || `서버 오류 (${res.status})`);
151 151     return data;
152 152 }
153 153 async function runFirst() { // entry: first analyze of a fresh capture
154 154     hideAll();
155 155     $("loadMsg").textContent = "신호를 분석하고 있어요...";
156 156     show($("loading"));
157 157     try {
158 158         const resp = await postTo(API, state.file, state.meta);
159 159         state.overview = resp.overview || null;
160 160         state.resp = resp;
161 161         render(resp);
162 162         if (state.overview) renderBurstMap(state.overview, resp);
163 163     } catch (e) {
164 164         showError(e.message);
165 165     }
166 166 }

```

```

167 167   async function analyze() {                                     // burst re-selection within a single signal
168 168     hideAll();
169 169     $("loadMsg").textContent = "신호를 분석하고 있어요...";
170 170     show($("loading"));
171 171     try {
172 172         state.resp = await postTo(API, state.file, state.meta, state.burst);
173 173         render(state.resp);
174 174         if (state.overview) renderBurstMap(state.overview, state.resp);
175 175         $("backBatch").classList.toggle("hidden", !state.batch.length);
176 176     } catch (e) {
177 177         showError(e.message);
178 178     }
179 179 }
180 180
181 181 /* --- batch mode --- */
182 182 async function runBatch(data, metas) {
183 183     hideAll();
184 184     show($("loading"));
185 185     const truths = new Map(metas.filter((m) => /\s.json$/i.test(m.name))
186 186         .map((m) => [m.name.replace(/\s.json$/i, ""), m]));
187 187     const sigmfs = new Map(metas.filter((m) => /\s.sigmf-meta$/i.test(m.name))
188 188         .map((m) => [m.name.replace(/\s.sigmf-meta$/i, ""), m]));
189 189     const rows = [];
190 190     for (let i = 0; i < data.length; i++) {
191 191         const f = data[i];
192 192         $("loadMsg").textContent = `일괄 분석 중... (${i + 1}/${data.length}) ${f.name}`;
193 193         // same meta precedence as single-file mode: a .sigmf-meta sidecar (matched by stem)
194 194         // beats filename tokens
195 195         const sm = sigmfs.get(f.name.replace(/\s[^\s]+$/, ""));
196 196         const meta = (sm && metaFromSigmf(await readJson(sm))) || parseName(f.name);
197 197         let resp = null, err = null;
198 198         try {
199 199             if (!meta.fs || !meta.fmt) throw new Error("파일명에 fs/포맷 정보가 없어요");
200 200             resp = await postTo(API, f, meta);
201 201         } catch (e) { err = e.message; }
202 202         const tf = truths.get(f.name);
203 203         // meta is stored per row: a later burst-chip re-analyze posts with THIS file's meta --
204 204         // it used to read state.meta, which a fresh batch never set (crash) and a previous
205 205         // single-file run left stale (silent wrong-fs decode)
206 206         rows.push({ file: f, meta, resp, err, sidecar: tf ? await readJson(tf) : null });
207 207     }
208 208     state.batch = rows;
209 209     hideAll();
210 210     renderBatch(rows);
211 211 }
212 212
213 213 function renderBatch(rows) {
214 214     $("batchBody").innerHTML = rows.map((r, i) => {
215 215         if (r.err) return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
216 216             `<td colspan="4" style="color:var(--red)">${r.err}</td></tr>`;
217 217         const d = r.resp.detected, lock = Math.round(r.resp.quality.lock);
218 218         const cls = lock >= 60 ? "ok" : lock >= 40 ? "warn" : "bad";
219 219         return `<tr data-i="${i}"><td class="fn">${r.file.name}</td>` +
220 220             `<td>${d.mod.toUpperCase()}</td><td>${fmtHz(d.fc)}</td><td>${fmtHz(d.baud)}</td>` +
221 221             `<td><span class="pill ${cls}">${lock}</span></td></tr>`;
222 222     }).join("");
223 223     $("batchBody").querySelectorAll("tr").forEach((tr) => {
224 224         tr.onclick = () => {
225 225             const r = state.batch[+tr.dataset.i];
226 226             if (!r.resp) return;

```

```

227 227 state.file = r.file; state.meta = r.meta; state.resp = r.resp; state.sidecar = r.sidecar;
228 228 state.burst = null; state.overview = null;
229 229 hideAll();
230 230 render(r.resp);
231 231 const b = $("backBatch");
232 232 b.textContent = "← 목록으로";
233 233 b.onclick = () => { hideAll(); renderBatch(state.batch); };
234 234 b.classList.remove("hidden");
235 235 };
236 236 });
237 237 show($("batchCard"));
238 238 }
239 239 $("backBatch").onclick = () => { hideAll(); renderBatch(state.batch); };
240 240 $("dlCsv").onclick = () => {
241 241     const head = "file,mod,fc_hz,baud_hz,rolloff,lock,mer_db,snr_est_db\n";
242 242     const body = state.batch.filter((r) => r.resp).map((r) => {
243 243         const d = r.resp.detected, q = r.resp.quality;
244 244         return [r.file.name, d.mod, d.fc, d.baud, d.rolloff ?? "", q.lock, q.mer_db,
245 245             r.resp.snr_est_db ?? ""].join(",");
246 246     }).join("\n");
247 247     saveBlob("signus_batch.csv", head + body, "text/csv");
248 248 };
249 249
250 250 /* --- burst map: whole-record spectrogram strip with clickable time-burst boxes --- */
251 251 function renderBurstMap(ov, result) {
252 252     show($("burstMap"));
253 253     const n = ov.n || 1, bs = result.bursts || [];
254 254     paintStrip($("burstFall"), ov.strip);
255 255     const host = $("burstBoxes");
256 256     host.innerHTML = "";
257 257     bs.forEach((b, i) => {
258 258         const left = Math.max(0, b.start / n * 100);
259 259         const dur = (b.end - b.start) / ov.fs;
260 260         const lbl = `버스트 ${i + 1} · ${dur >= 1 ? dur.toFixed(2) + " s" : (dur * 1e3).toFixed(0)
261 261             } + " ms"`;
262 262         const box = document.createElement("button"); // full height (one signal), spans t0..t1 i
263 263             n time
264 264         box.className = "box" + (i === result.burst_idx ? " sel" : "");
265 265         box.style.top = "0%"; box.style.height = "100%"; box.style.left = left + "%";
266 266         box.style.width = Math.min(100 - left, Math.max(0.8, (b.end - b.start) / n * 100)) + "%";
267 267         box.title = lbl;
268 268         box.innerHTML = `<span class="box-num">${i + 1}</span><span class="box-lane"></span>` +
269 269             `<span class="box-lbl">${lbl}</span>`;
270 270         box.onclick = () => { state.burst = i; analyze(); }; // re-analyse that burst; map persist
271 271             s
272 272         host.appendChild(box);
273 273     });
274 274 }
275 275
276 276 /* --- rendering --- */
277 277 function statusOf(lock) {
278 278     if (lock >= 60) return ["복조 성공", "ok"];
279 279     if (lock >= 40) return ["부분 복조", "warn"];
280 280     return ["복조 실패", "bad"];
281 281 }
282 282 function render(d) {
283 283     hideAll();
284 284     show($("results"));
285 285     $("burstMap").classList.add("hidden"); // re-shown by renderBurstMap only in multi-burst sin
286 286         gle mode

```

```

283 283   const lock = d.quality.lock;
284 284   let [txt, cls] = statusOf(lock);
285 285   if (d.detected.chirp) { txt = "처프 특성 보고"; cls = "warn"; }
286 286   const pill = $("statusPill");
287 287   pill.textContent = txt;
288 288   pill.className = "pill " + cls;
289 289   $("fileName").textContent =
290 290     `${state.file.name} · ${fmtHz(d.fs)} · ${d.fmt === "real" ? "Real PCM" : "IQ"}`;
291 291   $("merVal").textContent = d.quality.mer_db == null ? "-" : d.quality.mer_db.toFixed(1) + " dB"
292 292     ;
293 293   $("evmVal").textContent = d.quality.evm == null ? "-" : (d.quality.evm * 100).toFixed(1) + " %"
294 294     ;
295 295   $("snrVal").textContent = snrText(d);
296 296
297 297   const flags = [];
298 298   if (d.family === "fsk") flags.push('<span class="chip warn">FSK 계열 · 주파수 판별</span>');
299 299   if (d.family === "chirp") flags.push('<span class="chip warn">처프/CSS · 특성 판독</span>');
300 300   if (d.eq && d.eq.applied) flags.push('<span class="chip hit">등화기 적용 · ${
301 301     d.eq.mode === "fse" ? "T/2 분수간격" : "심볼간격"></span>');
302 302   if (d.detected.alias_resolved) flags.push('<span class="chip warn">반송파 앨리어스 보정</span>');
303 303   if (d.detected.baud_fallback) flags.push('<span class="chip warn">심볼레이트 대역폭 폴백</span>');
304 304   if (d.detected.carrier_ambiguous) flags.push('<span class="chip warn">반송파 모호 · 앨리어
305 305     싱 가능</span>');
306 306   $("flagRow").innerHTML = flags.join("");
307 307
308 308   renderBursts(d);
309 309   animateGauge(lock, cls);
310 310   renderParams(d);
311 311   drawSpectrum(d);
312 312   startPlay(d);
313 313 }
314 314
315 315 function snrText(d) {
316 316   const s = d.snr_est_db;
317 317   if (s == null || d.quality.lock < 30) return "-";
318 318   return (s > 28 ? "≥28" : s.toFixed(1)) + " dB";
319 319 }
320 320
321 321 function renderBursts(d) {
322 322   const row = $("burstRow"), bs = d.bursts || [];
323 323   // when the burst MAP is shown (multi-burst capture) the map replaces the chips
324 324   if (state.overview || bs.length < 2) { row.innerHTML = ""; return; }
325 325   const dur = (b) => {
326 326     const sec = (b.end - b.start) / d.fs;
327 327     return sec >= 1 ? sec.toFixed(2) + " s" : (sec * 1e3).toFixed(1) + " ms";
328 328   };
329 329   row.innerHTML = bs.map((b, i) =>
330 330     `<button class="chip-btn${i === d.burst_idx ? " on" : ""}" data-i="${i}">` +
331 331     `버스트 ${i + 1} · ${dur(b)}</button>`).join("");
332 332   row.querySelectorAll("button").forEach((el) => {
333 333     el.onclick = () => { state.burst = +el.dataset.i; analyze(); };
334 334   });
335 335 }
336 336
337 337 function chip(hit, truthTxt) {
338 338   return `<span class="chip ${hit ? "hit" : "miss"}">정답 ${truthTxt} · ${hit ? "일치" : "불일치"}</span>`;
339 339 }
340 340
341 341 function renderParams(d) {

```

```

338 337 const det = d.detected, t = state.sidecar && state.sidecar.truth;
339 338 const near = (a, b, tol) => Math.abs(a - b) <= tol;
340 339 if (det.chirp) { // 처프/CSS: 성상도 제원 대신 특성 (복조 없음, 판독만)
341 340   const c = det.chirp;
342 341   const crows = [
343 342     ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc), null,
344 343       det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
345 344     ["처프", `${c.up ? "▲ 상승" : "▼ 하강"} · ${(c.mu / 1e9).toFixed(3)} MHz/ms`, null,
346 345       `점유대역폭 ${fmtHz(c.bw)}`],
347 346     ["변조방식", c.sf ? `LoRa 추정 SF${c.sf}` : "처프(FMCW/CSS)", null,
348 347       c.sf ? `심볼레이트 ${fmtHz(c.rs)} · 심볼시간 ${(c.tsym * 1e3).toFixed(2)} ms` : ""],
349 348   ];
350 349   $("paramGrid").innerHTML = crows.map(paramCard).join("");
351 350   return;
352 351 }
353 352 const rows = [
354 353   ["중심주파수", det.rf_hz != null ? fmtRf(det.rf_hz) : fmtHz(det.fc),
355 354     t && t.fc != null && chip(near(det.fc, t.fc, Math.max(300, 0.02 * t.baud)), fmtHz(t.fc)),
356 355     det.rf_hz != null ? `기저대역 ${fmtHz(det.fc)}` : ""],
357 356   ["심볼레이트", fmtHz(det.baud), t && t.baud != null && chip(near(det.baud, t.baud, 0.03 * t.
358 357     baud), fmtHz(t.baud)),
359 358     det.baud_conf ? `스펙트럴 라인 강도 ×${Math.round(det.baud_conf)}` : ""],
360 359   ["변조방식", det.mod.toUpperCase(), t && t.mod != null && chip(det.mod === t.mod, t.mod.toUp
361 360     perCase()),
362 361     det.symmetry ? `대칭 ${det.symmetry}` : "주파수 레벨"],
363 362   ];
364 363   if (d.family === "fsk") {
365 364     rows.push(["변조지수 h", det.h.toFixed(2),
366 365       t && t.h != null && chip(near(det.h, t.h, 0.15), t.h.toFixed(2)), ""]);
367 366   } else {
368 367     rows.push(["롤오프", det.rolloff.toFixed(2),
369 368       t && t.rolloff != null && chip(near(det.rolloff, t.rolloff, 0.11), t.rolloff.toFixed(2))
370 369       , ""]);
371 370   }
372 371   $("paramGrid").innerHTML = rows.map(paramCard).join("");
373 372 }
374 373 function paramCard([label, val, hit, note]) {
375 374   return `
376 375     <div class="card param">
377 376       <div class="param-label">${label}</div>
378 377       <div class="param-val">${val}</div>
379 378       ${note ? `<div class="param-note">${note}</div>` : ""}
380 379       ${hit ? `<div class="chips">${hit}</div>` : ""}
381 380     </div>`;
382 381 }
383 382 function animateGauge(lock, cls) {
384 383   const arc = $("donutArc"), num = $("lockNum"), C = 2 * Math.PI * 52;
385 384   arc.style.stroke = { ok: "#12b886", warn: "#f59f00", bad: "#fa5252" }[cls];
386 385   arc.style.strokeDasharray = C;
387 386   const target = C * (1 - Math.max(0, Math.min(100, lock)) / 100);
388 387   if (REDUCED) { arc.style.strokeDashoffset = target; num.textContent = Math.round(lock); return
389 388     ; }
390 389   arc.style.strokeDashoffset = C;
391 390   requestAnimationFrame(() => { arc.style.strokeDashoffset = target; });
392 391   const t0 = performance.now();
393 392   const step = (t) => {
394     const k = Math.min(1, (t - t0) / 900);
395     num.textContent = Math.round(lock * (1 - Math.pow(1 - k, 3)));
  
```

```

394 393     if (k < 1) requestAnimationFrame(step);
395 394   };
396 395   requestAnimationFrame(step);
397 396 }
398 397
399 398 /* --- canvas helpers --- */
400 399 function fit(cv, hCss) {
401 400   const dpr = window.devicePixelRatio || 1, w = cv.clientWidth || 320;
402 401   const h = hCss || w;
403 402   cv.width = w * dpr; cv.height = h * dpr;
404 403   const g = cv.getContext("2d");
405 404   g.setTransform(dpr, 0, 0, dpr, 0, 0);
406 405   return [g, w, h];
407 406 }
408 407 function pct(a, p) {
409 408   const s = [...a].sort((x, y) => x - y);
410 409   return s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))];
411 410 }
412 411
413 412 /* --- spectrum --- */
414 413 function drawSpectrum(d) {
415 414   // sa 프로브의 power spectrum 판과 같은 조판: 흑배경, 백색 곡선, 회색(0.35) 점선 격자,
416 415   // 바닥 p5-2 .. 최대+2 dB. 검출된 중심주파수(실선)와 ±baud/2(점선)만 기능선으로 엮는다.
417 416   const [g, w, h] = fit($("#specCanvas"), 120);
418 417   g.fillStyle = "#000"; g.fillRect(0, 0, w, h);
419 418   if (!d.spectrum) return;
420 419   const f = d.spectrum.f, db = d.spectrum.db;
421 420   const lo = pct(db, 0.05) - 2, hi = pct(db, 1) + 2;
422 421   const fmin = f[0], fmax = f[f.length - 1];
423 422   const X = (v) => (v - fmin) / (fmax - fmin) * w;
424 423   const Y = (v) => h - (Math.max(lo, Math.min(hi, v)) - lo) / (hi - lo) * (h - 8) - 4;
425 424   g.strokeStyle = "rgba(255,255,255,.35)"; g.setLineDash([1, 5]);
426 425   for (let k = 1; k <= 4; k++) { // fs/2 의 k/5 지점 세로 격자 (sa 와 동일)
427 426     const gx = fmin + (fmax - fmin) * k / 5;
428 427     g.beginPath(); g.moveTo(X(gx), 0); g.lineTo(X(gx), h); g.stroke();
429 428   }
430 429   const det = d.detected, fca = Math.abs(det.fc) / 1e3, half = det.baud / 2e3;
431 430   g.setLineDash([4, 4]);
432 431   for (const gf of [fca - half, fca + half]) { // 단축(0..fs/2) 축 -- fc 는 접힌 절대값
433 432     g.beginPath(); g.moveTo(X(gf), 0); g.lineTo(X(gf), h); g.stroke();
434 433   }
435 434   g.setLineDash([]);
436 435   g.strokeStyle = "rgba(255,255,255,.6)";
437 436   g.beginPath(); g.moveTo(X(fca), 0); g.lineTo(X(fca), h); g.stroke();
438 437 + g.strokeStyle = "#4da3ff"; g.lineWidth = 1.4; g.beginPath(); // 파란선 (바늘이 주인공)
439 438   f.forEach((v, k) => (k ? g.lineTo(X(v), Y(db[k])) : g.moveTo(X(v), Y(db[k]))));
440 439   g.stroke();
441 440   g.fillStyle = "rgba(255,255,255,.6)"; g.font = "10px sans-serif";
442 441   g.fillText(sig(fmin) + " kHz", 4, h - 4);
443 442   const tmax = sig(fmax) + " kHz";
444 443   g.fillText(tmax, w - g.measureText(tmax).width - 4, h - 4);
445 444 }
446 445
447 446 + /* --- spectrogram strip (sa 프로브의 PNG 와 같은 조판, 버스트 지도 전용) --- */
448 447 function paintStrip(canvas, st) {
449 448   // 서버가 sa 와 같은 규칙(흑백 0..235, hamming 256/128, p25+10..35dB, 열 최대 폴링)으로
450 449   // 만든 격자를 해상도 그대로 찍는다. CSS 의 image-rendering: pixelated 가 sa PNG 를
451 450   // 확대해 보는 것과 같은 방식으로 늘린다 -- 보간 뭉개짐이 없다.
452 451   const g = canvas.getContext("2d");
453 452   canvas.width = st && st.cols ? st.cols : 1;
454 453

```

```

455 453 canvas.height = st && st.rows ? st.rows : 1;
456 454 if (!st || !st.cols) { g.fillStyle = "#000"; g.fillRect(0, 0, 1, 1); return; }
457 455 const img = g.createImageData(st.cols, st.rows);
458 456 for (let k = 0; k < st.g.length; k++) {
459 457     const o = k * 4;
460 458     img.data[o] = img.data[o + 1] = img.data[o + 2] = st.g[k];
461 459     img.data[o + 3] = 255;
462 460 }
463 461 g.putImageData(img, 0, 0);
464 462 }
465 463
466 464 /* --- constellation playback (the headline) --- */
467 465 const play = { raf: 0, i: 0, on: false, data: null };
468 466 let CG = null; // cached constellation geometry
469 467
470 468 function stopPlay() { cancelAnimationFrame(play.raf); play.raf = 0; play.on = false; }
471 469 function setPlayBtn(on) { $("playBtn").textContent = on ? "⏸ 일시정지" : "▶ 재생"; }
472 470
473 471 function startPlay(d) {
474 472     stopPlay();
475 473     play.data = d;
476 474     play.i = 0;
477 475     const n = d.constellation.i.length;
478 476     $("scrub").max = String(Math.max(1, n));
479 477     $("playInfo").textContent = `${n.toLocaleString()} 심볼 · 시간 순서로 재생 (잔광 효과)`;
480 478     drawConstBase(d);
481 479     if (REDUCED) { // static full view, no motion
482 480         drawPoints(d, 0, n);
483 481         $("scrub").value = String(n);
484 482         setPlayBtn(false);
485 483         return;
486 484     }
487 485     play.on = true;
488 486     setPlayBtn(true);
489 487     play.raf = requestAnimationFrame(loop);
490 488 }
491 489 $("playBtn").onclick = () => {
492 490     if (!play.data) return;
493 491     if (play.on) { stopPlay(); setPlayBtn(false); }
494 492     else { play.on = true; setPlayBtn(true); play.raf = requestAnimationFrame(loop); }
495 493 };
496 494 $("scrub").oninput = () => {
497 495     if (!play.data) return;
498 496     stopPlay(); setPlayBtn(false);
499 497     play.i = +$("scrub").value;
500 498     drawConstBase(play.data);
501 499     drawPoints(play.data, 0, play.i); // redraw history up to the scrub point
502 500 };
503 501
504 502 function loop() {
505 503     const d = play.data, n = d.constellation.i.length;
506 504     const step = Math.max(1, Math.round(n / 600) * (+$("speedSel").value));
507 505     const to = Math.min(n, play.i + step);
508 506     fade(); // phosphor persistence
509 507     drawPoints(d, play.i, to);
510 508     play.i = to >= n ? 0 : to;
511 509     if (play.i === 0) drawConstBase(d); // loop: clear the trail
512 510     $("scrub").value = String(play.i);
513 511     if (play.on) play.raf = requestAnimationFrame(loop);
514 512 }

```

```

515 513
516 514 function drawConstBase(d) {
517 515     const [g, w] = fit$("constCanvas");
518 516     const fsk = d.family === "fsk";
519 517     const I = d.constellation.i, Q = d.constellation.q;
520 518     const ideal = fsk ? [] : idealPoints(d.detected.mod);
521 519     let lim = 0;
522 520     for (let k = 0; k < I.length; k++) lim = Math.max(lim, Math.abs(I[k]), Math.abs(Q[k]));
523 521     for (const p of ideal) lim = Math.max(lim, Math.abs(p.i), Math.abs(p.q));
524 522     lim = (lim || 1) * 1.12;
525 523     const R = w / 2 * 0.9;
526 524     CG = { g, w, lim, R, map: (re, im) => [w / 2 + re / lim * R, w / 2 - im / lim * R] };
527 525
528 526     g.fillStyle = "#0d1320"; g.fillRect(0, 0, w, w);
529 527     g.strokeStyle = "rgba(255,255,255,.06)"; g.lineWidth = 1;
530 528     for (let f = -1; f <= 1; f += 0.5) {
531 529         const [gx] = CG.map(f, 0), [, gy] = CG.map(0, f);
532 530         g.beginPath(); g.moveTo(gx, 0); g.lineTo(gx, w); g.stroke();
533 531         g.beginPath(); g.moveTo(0, gy); g.lineTo(w, gy); g.stroke();
534 532     }
535 533     if (fsk) { // frequency levels as dashed bands
536 534         g.strokeStyle = "rgba(255,255,255,.5)"; g.setLineDash([6, 5]);
537 535         for (const lv of [...new Set(I.map((v) => Math.round(v)))] .filter((v) => Math.abs(v) <= 8)
538 536             ) {
539 537             const [, yy] = CG.map(0, lv);
540 538             g.beginPath(); g.moveTo(0, yy); g.lineTo(w, yy); g.stroke();
541 539         }
542 540         g.setLineDash([]);
543 541     } else {
544 542         g.strokeStyle = "rgba(255,255,255,.75)"; g.lineWidth = 1.4;
545 543         for (const p of ideal) {
546 544             const [x, y] = CG.map(p.i, p.q);
547 545             g.beginPath(); g.arc(x, y, 6, 0, 7); g.stroke();
548 546         }
549 547     }
550 548 function fade() {
551 549     if (!CG) return;
552 550     CG.g.fillStyle = "rgba(13,19,32,.14)"; // translucent wash = phosphor decay
553 551     CG.g.fillRect(0, 0, CG.w, CG.w);
554 552 }
555 553 function drawPoints(d, from, to) {
556 554     if (!CG || to <= from) return;
557 555     const g = CG.g, I = d.constellation.i, Q = d.constellation.q;
558 556     const fsk = d.family === "fsk";
559 557     const a = Math.max(0.08, Math.min(0.55, 150 / (to - from)));
560 558     g.globalCompositeOperation = "lighter";
561 559     g.fillStyle = `rgba(90,170,255,${a})`;
562 560     for (let k = from; k < to; k++) {
563 561         // FSK has no I/Q plane: spread levels vertically, jitter horizontally for density
564 562         const [x, y] = fsk ? CG.map((k % 89) / 89 - 0.5, I[k]) : CG.map(I[k], Q[k]);
565 563         g.beginPath(); g.arc(x, y, 1.8, 0, 7); g.fill();
566 564     }
567 565     g.globalCompositeOperation = "source-over";
568 566 }
569 567
570 568 /* --- downloads --- */
571 569 function saveBlob(name, text, type) {
572 570     const url = URL.createObjectURL(new Blob([text], { type }));
573 571     const a = document.createElement("a");

```

```

574 572   a.href = url; a.download = name; a.click();
575 573   URL.revokeObjectURL(url);
576 574 }
577 575 const stem = () => state.file.name.replace(/\.([^.]*$)/, "");
578 576 $("copyBits").onclick = async (e) => { // decoded bitstream -> clipboard, with brief feedback
579 577   const btn = e.currentTarget, was = btn.textContent;
580 578   try { await navigator.clipboard.writeText(state.resp.bits); btn.textContent = "복사됨 ✓"; }
581 579   catch { btn.textContent = "복사 실패"; }
582 580   setTimeout(() => { btn.textContent = was; }, 1400);
583 581 };
584 582 $("dlBits").onclick = () => saveBlob(stem() + ".bits.txt", state.resp.bits, "text/plain");
585 583 $("dlSyms").onclick = () => {
586 584   const c = state.resp.constellation;
587 585   saveBlob(stem() + ".symbols.csv", "i,q\n" + c.i.map((v, k) => `${v},${c.q[k]}`).join("\n"), "text/csv");
588 586 };
589 587 $("dlReport").onclick = () =>
590 588   saveBlob(stem() + ".report.json", JSON.stringify(state.resp, null, 2), "application/json");
591 589
592 590 /* --- view helpers --- */
593 591 function show(el) { el.classList.remove("hidden"); }
594 592 function hideAll() {
595 593   stopPlay();
596 594   ["metaForm", "loading", "errorCard", "results", "batchCard"]
597 595     .forEach((id) => $(id).classList.add("hidden"));
598 596   $("backBatch").classList.add("hidden");
599 597 }
600 598 function showError(msg) { hideAll(); $("errMsg").textContent = msg; show($("errorCard")); }
601 599
602 600 /* --- dropzone wiring --- */
603 601 const drop = $("dropCard"), input = $("fileInput");
604 602 // reset value first: re-picking the SAME file fires no change event otherwise
605 603 const pick = () => { input.value = ""; input.click(); };
606 604 drop.onclick = pick;
607 605 drop.onkeydown = (e) => { if (e.key === "Enter" || e.key === " ") { e.preventDefault(); pick()
608 606   ; } };
609 607 input.onChange = () => { if (input.files.length) acceptFiles(input.files); };
610 608 drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("drag"); });
611 609 drop.addEventListener("dragleave", () => drop.classList.remove("drag"));
612 610 drop.addEventListener("drop", (e) => {
613 611   e.preventDefault(); drop.classList.remove("drag");
614 612   if (e.dataTransfer.files.length) acceptFiles(e.dataTransfer.files);
   });

```